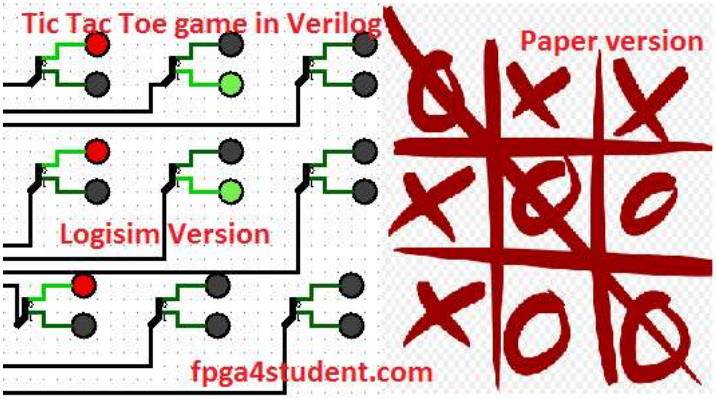
Tic Tac Toe Game in Verilog and LogiSim

Tic Tac Toe is a very popular paper-and-pencil game in a 3x3 grid for two players. The player who makes the first three of their marks in a diagonal, vertical, or horizontal row wins the game.

Today, fpga4student designs and implements the Tic Tac Toe game in Verilog and Logisim.



Firstly, the Tic Tac Toe game is designed and implemented in Logisim. However, let's define the rules for the game at first. In this game, a player plays the Tic Tac Toe game with a computer. When the player/ computer plays the game, a 2-bit value is stored into one of the nine positions in the 3x3 grid like Xs/ Os in the real paper-and-pencil version. 2'b00 is stored into a position when neither the player or computer played in that position. Similarly, 2'b01 (X) is the value to be stored when the player played in the position and 2'b10 (O) is the value to be saved when the computer played in the position. The player/ computer plays the game by pressing their corresponding button. Red/ Green LED is lit in a position when the position is played by the player/ computer respectively.

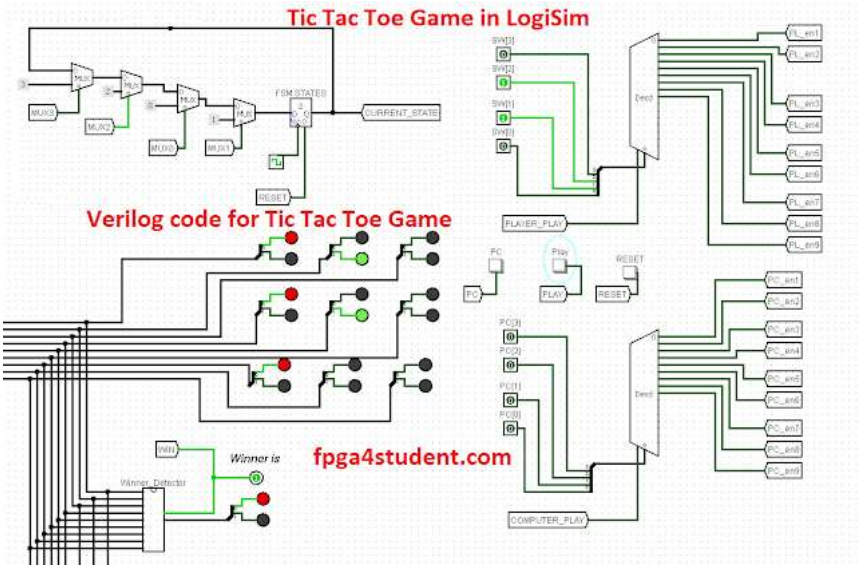
Consider the 3x3 grid table below as the order of the positions being played:

1	2	3
4	5	6
7	8	9

fpga4student.com

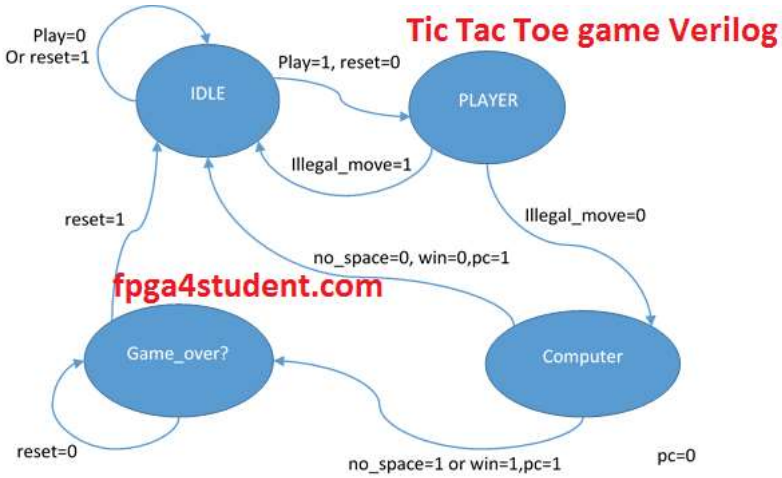
The player/ computer wins the game when successfully placing three similar (01-Xs) or (10-Os) values in the following row pairs: (1,2,3); (4,5,6);(7,8,9); (1,4,7); (2,5,8); (3,6,9); (1,5,9);(3,5,7).

The winner detecting circuit is designed to find the winner when the above winning rule is matched. To detect an illegal move, a comparator is needed to check if the current position was already played by either the computer or player. Moreover, “No space” detector is to check if all the positions are played and no winner is found.



You can download the full Logisim design for the Tic Tac Toe game here.

Let's go for the Verilog code of the Tic Tac Toe game. To control the Tic Tac Toe game, a FSM controller is designed as follows.



1. IDLE(00): when waiting for the player/ computer to play or when resetting the circuit, the FSM is at the IDLE state.

2. PLAYER(01): The player turns to play and "01" to be stored into the decoded position.

3. COMPUTER(10):
The computer turns to play and "01" to be stored into the decoded position.

4. Game_over(11): The game is finished when there is a winner or no more space to play.

Inputs of the controller of the Tic Tac Toe game:

a. Reset :

Reset = 1: Reset the game when in the Game_Over state.

Reset = 0: The game begins.

b. Play:

Play = 1: When in the IDLE state, play = 1 is to switch the controller to the PLAYER state and the player plays.

Play =0: Stay in the IDLE state.

c. PC

PC = 1: When in COMPUTER state, PC = 1 is to switch to the IDLE state and the computer plays.

PC =0 : stay in COMPUTER state.

d. Illegal_move

Illegal_move = 0: When in PLAYER state, Illegal_move = 0 is to switch to COMPUTER state and let computer plays when PC = 1.

Illegal_move = 1: Illegal moving from the player/ computer and switch to the IDLE state.
e. No_space
No_space = 0: still have space to play, continue the game.
No_space = 1: no more space to play, game over, and need to reset the game before playing again.
f. Win
Win = 0: Still waiting for the winner
Win = 1: There is a winner, finish the game, and need to reset the game before playing again.

Verilog code for the Tic Tac Toe game:

```
// fpga4student.com: FPGA projects, Verilog projects, VHDL projects
// Verilog code for TIC TAC TOE GAME
// Top level module
module tic_tac_toe_game(
     input clock, // clock of the game
     input reset, // reset button to reset the game
     input play, // play button to enable player to play
     input pc, // pc button to enable computer to play
     input [3:0] computer_position,player_position,
     // positions to play
     output wire [1:0] pos1,pos2,pos3,
     pos4,pos5,pos6,pos7,pos8,pos9,
     // LED display for positions
     // 01: Player
     // 10: Computer
     output wire[1:0]who
     // who the winner is
     );
 wire [15:0] PC_en;// Computer enable signals
 wire [15:0] PL_en; // Player enable signals
 wire illegal_move; // disable writing when an illegal move is detected
 //wire [1:0] pos1,pos2,pos3,pos4,pos5,pos6,pos7,pos8,pos9;// positions stored
 wire win; // win signal
 wire computer_play; // computer enabling signal
 wire player_play; // player enabling signal
 wire no_space; // no space signal
```

```
 // position registers
  position_registers position_reg_unit(
      clock, // clock of the game
      reset, // reset the game
      illegal_move, // disable writing when an illegal move is detected
      PC_en[8:0], // Computer enable signals
      PL_en[8:0], // Player enable signals
      pos1,pos2,pos3,pos4,pos5,pos6,pos7,pos8,pos9// positions stored
      );
 // winner detector
 winner_detector win_detect_unit(pos1,pos2,pos3,pos4,pos5,pos6,pos7,pos8,pos9,win,
 // position decoder for computer
 position_decoder pd1(computer_position,computer_play,PC_en);
 // position decoder for player
 position_decoder pd2(player_position,player_play,PL_en);
 // illegal move detector
  illegal_move_detector imd_unit(
   pos1,pos2,pos3,pos4,pos5,pos6,pos7,pos8,pos9,
   PC_en[8:0], PL_en[8:0],
   illegal_move
   );
 // no space detector
 nospace_detector nsd_unit(
   pos1,pos2,pos3,pos4,pos5,pos6,pos7,pos8,pos9,
   no_space
    );
 fsm_controller tic_tac_toe_controller(
     clock,// clock of the circuit
     reset,// reset
     play, // player plays
     pc,// computer plays
     illegal_move,// illegal move detected
     no_space, // no_space detected
     win, // winner detected
     computer_play, // enable computer to play
     player_play // enable player to play
     );
endmodule
// Position registers
// fpga4student.com: FPGA projects, Verilog projects, VHDL projects
// to store player or computer positions
// when enabling by the FSM controller
module position_registers(
      input clock, // clock of the game
      input reset, // reset the game
      input illegal_move, // disable writing when an illegal move is detected
      input [8:0] PC_en, // Computer enable signals
      input [8:0] PL_en, // Player enable signals
      output reg[1:0] pos1,pos2,pos3,pos4,pos5,pos6,pos7,pos8,pos9// positions sto
      );
 // Position 1
 always @(posedge clock or posedge reset)
 begin
  if(reset)
   pos1 <= 2'b00;
  else begin
   if(illegal_move==1'b1)
    pos1 <= pos1;// keep previous position
   else if(PC_en[0]==1'b1)
    pos1 <= 2'b10; // store computer data
   else if (PL_en[0]==1'b1)
    pos1 <= 2'b01;// store player data
   else
```

```
    pos1 <= pos1;// keep previous position
   end
  end
  // Position 2
  always @(posedge clock or posedge reset)
  begin
   if(reset)
    pos2 <= 2'b00;
   else begin
    if(illegal_move==1'b1)
     pos2 <= pos2;// keep previous position
    else if(PC_en[1]==1'b1)
     pos2 <= 2'b10; // store computer data
    else if (PL_en[1]==1'b1)
     pos2 <= 2'b01;// store player data
    else
     pos2 <= pos2;// keep previous position
   end
  end
 // Position 3
  always @(posedge clock or posedge reset)
  begin
   if(reset)
    pos3 <= 2'b00;
   else begin
    if(illegal_move==1'b1)
     pos3 <= pos3;// keep previous position
    else if(PC_en[2]==1'b1)
     pos3 <= 2'b10; // store computer data
    else if (PL_en[2]==1'b1)
     pos3 <= 2'b01;// store player data
    else
     pos3 <= pos3;// keep previous position
   end
  end
  // Position 4
  always @(posedge clock or posedge reset)
  begin
   if(reset)
    pos4 <= 2'b00;
   else begin
    if(illegal_move==1'b1)
     pos4 <= pos4;// keep previous position
    else if(PC_en[3]==1'b1)
     pos4 <= 2'b10; // store computer data
    else if (PL_en[3]==1'b1)
     pos4 <= 2'b01;// store player data
    else
     pos4 <= pos4;// keep previous position
   end
  end
  // Position 5
  always @(posedge clock or posedge reset)
  begin
   if(reset)
    pos5 <= 2'b00;
   else begin
    if(illegal_move==1'b1)
     pos5 <= pos5;// keep previous position
    else if(PC_en[4]==1'b1)
     pos5 <= 2'b10; // store computer data
    else if (PL_en[4]==1'b1)
     pos5 <= 2'b01;// store player data
```

```
   else
    pos5 <= pos5;// keep previous position
  end
 end
 // Position 6
 always @(posedge clock or posedge reset)
 begin
  if(reset)
   pos6 <= 2'b00;
  else begin
   if(illegal_move==1'b1)
    pos6 <= pos6;// keep previous position
   else if(PC_en[5]==1'b1)
    pos6 <= 2'b10; // store computer data
   else if (PL_en[5]==1'b1)
    pos6 <= 2'b01;// store player data
   else
    pos6 <= pos6;// keep previous position
  end
 end
 // Position 7
 always @(posedge clock or posedge reset)
 begin
  if(reset)
   pos7 <= 2'b00;
  else begin
   if(illegal_move==1'b1)
    pos7 <= pos7;// keep previous position
   else if(PC_en[6]==1'b1)
    pos7 <= 2'b10; // store computer data
   else if (PL_en[6]==1'b1)
    pos7 <= 2'b01;// store player data
   else
    pos7 <= pos7;// keep previous position
  end
 end
 // Position 8
 always @(posedge clock or posedge reset)
 begin
  if(reset)
   pos8 <= 2'b00;
  else begin
   if(illegal_move==1'b1)
    pos8 <= pos8;// keep previous position
   else if(PC_en[7]==1'b1)
    pos8 <= 2'b10; // store computer data
   else if (PL_en[7]==1'b1)
    pos8 <= 2'b01;// store player data
   else
    pos8 <= pos8;// keep previous position
  end
 end
 // Position 9
 always @(posedge clock or posedge reset)
 begin
  if(reset)
   pos9 <= 2'b00;
  else begin
   if(illegal_move==1'b1)
    pos9 <= pos9;// keep previous position
   else if(PC_en[8]==1'b1)
    pos9 <= 2'b10; // store computer data
   else if (PL_en[8]==1'b1)
```

```
    pos9 <= 2'b01;// store player data
   else
    pos9 <= pos9;// keep previous position
  end
 end
endmodule
// FSM controller to control how player and computer play the TIC TAC TOE GAME
// The FSM is implemented based on the designed state diagram
// fpga4student.com: FPGA projects, Verilog projects, VHDL projects
module fsm_controller(
     input clock,// clock of the circuit
     input reset,// reset
     play, // player plays
     pc,// computer plays
     illegal_move,// illegal move detected
     no_space, // no_space detected
     win, // winner detected
     output reg computer_play, // enable computer to play
     player_play // enable player to play
     );
// FSM States
parameter IDLE=2'b00;
parameter PLAYER=2'b01;
parameter COMPUTER=2'b10;
parameter GAME_DONE=2'b11;
reg[1:0] current_state, next_state;
// current state registers
always @(posedge clock or posedge reset)
begin
 if(reset)
  current_state <= IDLE;
 else
  current_state <= next_state;
end
 // next state
always @(*)
 begin
 case(current_state)
 IDLE: begin
  if(reset==1'b0 && play == 1'b1)
   next_state <= PLAYER; // player to play
  else
   next_state <= IDLE;
  player_play <= 1'b0;
  computer_play <= 1'b0;
 end
 PLAYER:begin
  player_play <= 1'b1;
  computer_play <= 1'b0;
  if(illegal_move==1'b0)
   next_state <= COMPUTER; // computer to play
  else
   next_state <= IDLE;
 end
 COMPUTER:begin
  player_play <= 1'b0;
  if(pc==1'b0) begin
   next_state <= COMPUTER;
   computer_play <= 1'b0;
  end
  else if(win==1'b0 && no_space == 1'b0)
  begin
   next_state <= IDLE;
```

```
    computer_play <= 1'b1;// computer to play when PC=1
   end
   else if(no_space == 1 || win ==1'b1)
   begin
    next_state <= GAME_DONE; // game done
    computer_play <= 1'b1;// computer to play when PC=1
   end
  end
  GAME_DONE:begin // game done
   player_play <= 1'b0;
   computer_play <= 1'b0;
   if(reset==1'b1)
    next_state <= IDLE;// reset the game to IDLE
   else
    next_state <= GAME_DONE;
  end
  default: next_state <= IDLE;
  endcase
  end
endmodule
// NO SPACE detector
// to detect if no more spaces to play
// fpga4student.com: FPGA projects, Verilog projects, VHDL projects
module nospace_detector(
   input [1:0] pos1,pos2,pos3,pos4,pos5,pos6,pos7,pos8,pos9,
   output wire no_space
    );
wire temp1,temp2,temp3,temp4,temp5,temp6,temp7,temp8,temp9;
// detect no more space
assign temp1 = pos1[1] | pos1[0];
assign temp2 = pos2[1] | pos2[0];
assign temp3 = pos3[1] | pos3[0];
assign temp4 = pos4[1] | pos4[0];
assign temp5 = pos5[1] | pos5[0];
assign temp6 = pos6[1] | pos6[0];
assign temp7 = pos7[1] | pos7[0];
assign temp8 = pos8[1] | pos8[0];
assign temp9 = pos9[1] | pos9[0];
// output
assign no_space =((((((((temp1 & temp2) & temp3) & temp4) & temp5) & temp6) & temp
endmodule

// Illegal move detector
// to detect if a player plays on an exist position
// fpga4student.com: FPGA projects, Verilog projects, VHDL projects
module illegal_move_detector(
   input [1:0] pos1,pos2,pos3,pos4,pos5,pos6,pos7,pos8,pos9,
   input [8:0] PC_en, PL_en,
   output wire illegal_move
   );
wire temp1,temp2,temp3,temp4,temp5,temp6,temp7,temp8,temp9;
wire temp11,temp12,temp13,temp14,temp15,temp16,temp17,temp18,temp19;
wire temp21,temp22;
// player : illegal moving
assign temp1 = (pos1[1] | pos1[0]) & PL_en[0];
assign temp2 = (pos2[1] | pos2[0]) & PL_en[1];
assign temp3 = (pos3[1] | pos3[0]) & PL_en[2];
assign temp4 = (pos4[1] | pos4[0]) & PL_en[3];
assign temp5 = (pos5[1] | pos5[0]) & PL_en[4];
assign temp6 = (pos6[1] | pos6[0]) & PL_en[5];
assign temp7 = (pos7[1] | pos7[0]) & PL_en[6];
assign temp8 = (pos8[1] | pos8[0]) & PL_en[7];
assign temp9 = (pos9[1] | pos9[0]) & PL_en[8];
```

```
// computer : illegal moving
assign temp11 = (pos1[1] | pos1[0]) & PC_en[0];
assign temp12 = (pos2[1] | pos2[0]) & PC_en[1];
assign temp13 = (pos3[1] | pos3[0]) & PC_en[2];
assign temp14 = (pos4[1] | pos4[0]) & PC_en[3];
assign temp15 = (pos5[1] | pos5[0]) & PC_en[4];
assign temp16 = (pos6[1] | pos6[0]) & PC_en[5];
assign temp17 = (pos7[1] | pos7[0]) & PC_en[6];
assign temp18 = (pos8[1] | pos8[0]) & PC_en[7];
assign temp19 = (pos9[1] | pos9[0]) & PC_en[8];
// intermediate signals
assign temp21 =(((((((temp1 | temp2) | temp3) | temp4) | temp5) | temp6) | temp7)
assign temp22 =((((((((temp11 | temp12) | temp13) | temp14) | temp15) | temp16) |
// output illegal move
assign illegal_move = temp21 | temp22 ;
endmodule
// fpga4student.com: FPGA projects, Verilog projects, VHDL projects
// To decode the position being played, a 4-to-16 decoder with high active output
// When a button is pressed, a player will play and the position at IN [3:0] will
// to enable writing to the corresponding registers
module position_decoder(input[3:0] in, input enable, output wire [15:0] out_en);
 reg[15:0] temp1;
 assign out_en = (enable==1'b1)?temp1:16'd0;
 always @(*)
 begin
 case(in)
 4'd0: temp1 <= 16'b0000000000000001;
 4'd1: temp1 <= 16'b0000000000000010;
 4'd2: temp1 <= 16'b0000000000000100;
 4'd3: temp1 <= 16'b0000000000001000;
 4'd4: temp1 <= 16'b0000000000010000;
 4'd5: temp1 <= 16'b0000000000100000;
 4'd6: temp1 <= 16'b0000000001000000;
 4'd7: temp1 <= 16'b0000000010000000;
 4'd8: temp1 <= 16'b0000000100000000;
 4'd9: temp1 <= 16'b0000001000000000;
 4'd10: temp1 <= 16'b0000010000000000;
 4'd11: temp1 <= 16'b0000100000000000;
 4'd12: temp1 <= 16'b0001000000000000;
 4'd13: temp1 <= 16'b0010000000000000;
 4'd14: temp1 <= 16'b0100000000000000;
 4'd15: temp1 <= 16'b1000000000000000;
 default: temp1 <= 16'b0000000000000001;
 endcase
end
endmodule
// fpga4student.com: FPGA projects, Verilog projects, VHDL projects
// winner detector circuit
// to detect who the winner is
// We will win when we have 3 similar (x) or (O) in the following pairs:
// (1,2,3); (4,5,6);(7,8,9); (1,4,7); (2,5,8);(3,6,9); (1,5,9);(3,5,7);
module winner_detector(input [1:0] pos1,pos2,pos3,pos4,pos5,pos6,pos7,pos8,pos9, o
wire win1,win2,win3,win4,win5,win6,win7,win8;
wire [1:0] who1,who2,who3,who4,who5,who6,who7,who8;
winner_detect_3 u1(pos1,pos2,pos3,win1,who1);// (1,2,3);
winner_detect_3 u2(pos4,pos5,pos6,win2,who2);// (4,5,6);
winner_detect_3 u3(pos7,pos8,pos9,win3,who3);// (7,8,9);
winner_detect_3 u4(pos1,pos4,pos7,win4,who4);// (1,4,7);
winner_detect_3 u5(pos2,pos5,pos8,win5,who5);// (2,5,8);
winner_detect_3 u6(pos3,pos6,pos9,win6,who6);// (3,6,9);
winner_detect_3 u7(pos1,pos5,pos9,win7,who7);// (1,5,9);
winner_detect_3 u8(pos3,pos5,pos6,win8,who8);// (3,5,7);
assign winner = (((((((win1 | win2) | win3) | win4) | win5) | win6) | win7) | win8
```

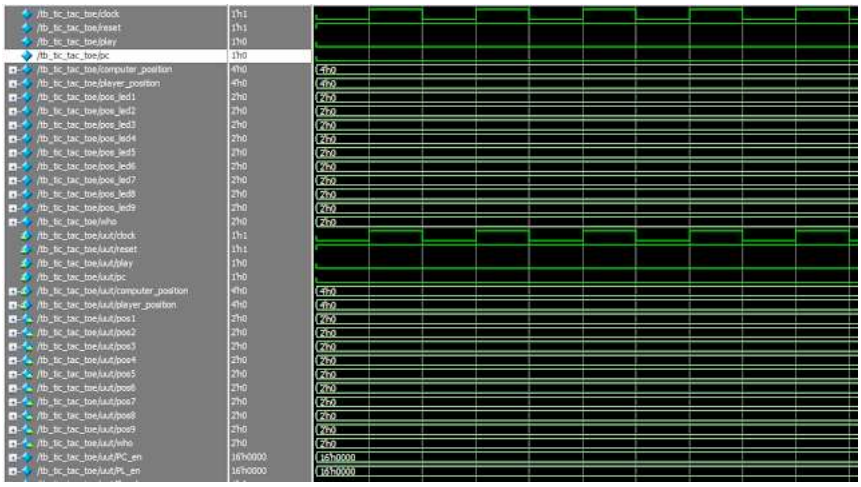
```
assign who = (((((((who1 | who2) | who3) | who4) | who5) | who6) | who7) | who8);
endmodule
// fpga4student.com: FPGA projects, Verilog projects, VHDL projects
// winner detection for 3 positions and determine who the winner is
// Player: 01
// Computer: 10
module winner_detect_3(input [1:0] pos0,pos1,pos2, output wire winner, output wire
wire [1:0] temp0,temp1,temp2;
wire temp3;
assign temp0[1] = !(pos0[1]^pos1[1]);
assign temp0[0] = !(pos0[0]^pos1[0]);
assign temp1[1] = !(pos2[1]^pos1[1]);
assign temp1[0] = !(pos2[0]^pos1[0]);
assign temp2[1] = temp0[1] & temp1[1];
assign temp2[0] = temp0[0] & temp1[0];
assign temp3 = pos0[1] | pos0[0];
// winner if 3 positions are similar and should be 01 or 10
assign winner = temp3 & temp2[1] & temp2[0];
// determine who the winner is
assign who[1] = winner & pos0[1];
assign who[0] = winner & pos0[0];
endmodule
```

Verilog testbench code for the Tic Tac Toe game:

```
`timescale 1ns / 1ps
// fpga4student.com: FPGA projects, Verilog projects, VHDL projects
// Verilog testbench code for TIC TAC TOE GAME

module tb_tic_tac_toe;

 // Inputs
 reg clock;
 reg reset;
 reg play;
 reg pc;
 reg [3:0] computer_position;
 reg [3:0] player_position;

 // Outputs
 wire [1:0] pos_led1;
 wire [1:0] pos_led2;
 wire [1:0] pos_led3;
 wire [1:0] pos_led4;
 wire [1:0] pos_led5;
 wire [1:0] pos_led6;
 wire [1:0] pos_led7;
 wire [1:0] pos_led8;
 wire [1:0] pos_led9;
 wire [1:0] who;

 // Instantiate the Unit Under Test (UUT)
 tic_tac_toe_game uut (
  .clock(clock),
  .reset(reset),
  .play(play),
  .pc(pc),
  .computer_position(computer_position),
  .player_position(player_position),
  .pos1(pos_led1),
  .pos2(pos_led2),
  .pos3(pos_led3),
  .pos4(pos_led4),
  .pos5(pos_led5),
```

```
  .pos6(pos_led6),
  .pos7(pos_led7),
  .pos8(pos_led8),
  .pos9(pos_led9),
  .who(who)
 );
 // clock
 initial begin
 clock = 0;
 forever #5 clock = ~clock;
 end
 initial begin
  // Initialize Inputs
  play = 0;
  reset = 1;
  computer_position = 0;
  player_position = 0;
  pc = 0;
  #100;
  reset = 0;
  #100;
  play = 1;
  pc = 0;
  computer_position = 4;
  player_position = 0;
  #50;
  pc = 1;
  play = 0;
  #100;
  reset = 0;
  play = 1;
  pc = 0;
  computer_position = 8;
  player_position = 1;
  #50;
  pc = 1;
  play = 0;
  #100;
  reset = 0;
  play = 1;
  pc = 0;
  computer_position = 6;
  player_position = 2;
  #50;
  pc = 1;
  play = 0;
  #50
  pc = 0;
  play = 0;
  end

endmodule
```

Simulation Waveform for the Tic Tac Toe game:



Recommended Verilog projects:

1. **What is an FPGA? How Verilog works on FPGA**
2. Verilog code for FIFO memory
3. Verilog code for 16-bit single-cycle MIPS processor
4. Programmable Digital Delay Timer in Verilog HDL
5. Verilog code for basic logic components in digital circuits
6. Verilog code for 32-bit Unsigned Divider
7. Verilog code for Fixed-Point Matrix Multiplication
8. Plate License Recognition in Verilog HDL
9. Verilog code for Carry-Look-Ahead Multiplier
10. Verilog code for a Microcontroller
11. Verilog code for 4x4 Multiplier
12. Verilog code for Car Parking System
13. Image processing on FPGA using Verilog HDL
14. How to load a text file into FPGA using Verilog HDL
15. Verilog code for Traffic Light Controller
16. Verilog code for Alarm Clock on FPGA
17. Verilog code for comparator design
18. Verilog code for D Flip Flop
19. Verilog code for Full Adder
20. Verilog code for counter with testbench
21. Verilog code for 16-bit RISC Processor
22. Verilog code for button debouncing on FPGA
23. How to write Verilog Testbench for bidirectional/ inout ports
24. Tic Tac Toe Game in Verilog and LogiSim
25. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)
26. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
27. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
28. Verilog code for Decoder
29. Verilog code for Multiplexers
30. N-bit Adder Design in Verilog
31. Verilog vs VHDL: Explain by Examples
32. Verilog code for Clock divider on FPGA
33. **How to generate a clock enable signal in Verilog**
34. **Verilog code for PWM Generator**
35. **Verilog coding vs Software Programming**
36. **Verilog code for Sequence Detector using Moore FSM**

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.